AQI Forecasting System for Karachi

An End-to-End Machine Learning Pipeline for Air Quality Index Prediction

Prepared by

Uroosham Memon

10Pearls Internship Project

Project Report

Live Project Link: https://aqi-forecasting-10pearls.streamlit.app/

GitHub Repository: https://github.com/UrooshamMemon/AQI_Quality_Index

August 2026

Table of Contents

1. Introduction

1.1 Project Background

Air pollution is a major environmental concern that can negatively affect human health and quality of life. The Air Quality Index (AQI) is commonly used to represent the overall level of air pollution in a simplified numerical form. AQI is influenced by different pollutants, including PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone, as well as weather conditions such as temperature, humidity, pressure, and wind speed.

With the availability of real-time environmental data and machine learning techniques, it is possible to analyze historical air-quality patterns and develop predictive systems that forecast future AQI values. This project focuses on developing an end-to-end AQI forecasting system for Karachi, combining automated data collection, feature engineering, machine learning, cloud-based storage, model management, automated pipelines, and an interactive web dashboard.

1.2 Problem Statement

Traditional AQI monitoring primarily provides information about the current or past state of air quality. However, knowing the expected air quality in advance can provide additional value for monitoring and decision-making.

The objective of this project is to develop a system capable of forecasting AQI for the next 24, 48, and 72 hours using historical AQI, pollutant, and weather data. The system must also automate data collection and processing, store features in a cloud-based Feature Store, train and manage forecasting models, and present current and predicted AQI values through an interactive dashboard.

1.3 Project Motivation

The motivation for this project is to explore how machine learning and automated data pipelines can be applied to environmental data to produce useful air-quality forecasts.

The project was also intended to provide practical experience with an end-to-end data science and machine learning workflow, including API integration, data preprocessing, feature engineering, model experimentation, model evaluation, cloud Feature Store management, Model Registry, CI/CD automation, explainability, and dashboard deployment.

1.4 Project Objectives

The main objectives of the project were to:

- Collect real-time and historical weather and air-quality data from external APIs.
- Develop a feature-engineering pipeline for processing environmental data.
- Create historical training data through data backfilling.
- Store processed features in the Hopsworks Feature Store.
- Develop separate machine learning models for 24-hour, 48-hour, and 72-hour AQI forecasting.
- Experiment with multiple machine learning algorithms and evaluate them using MAE, RMSE, and R^2.
- Compare forecasting models against persistence baselines.
- Register the selected models in the Hopsworks Model Registry.
- Automate feature collection and model retraining through scheduled pipelines.
- Develop an interactive Streamlit dashboard displaying current AQI, forecasts, pollutant levels, historical trends, EDA, alerts, and SHAP-based explainability.
- Deploy the dashboard so that the complete system can be accessed through a web interface.

1.5 Project Scope

The scope of the project covers the development of an end-to-end AQI forecasting system for Karachi.

The forecasting component focuses on 24-hour, 48-hour, and 72-hour AQI predictions. The project uses externally available environmental data through APIs and does not include the deployment of physical air-quality sensors or the development of a custom AQI measurement device.

The final system is designed as a complete machine learning pipeline, from data collection and feature engineering to model training, prediction, automation, and visualization.

2. Project Requirements

2.1 Functional Requirements

- Collect weather and air-quality data from external APIs.
- Process and engineer features for AQI forecasting.
- Store and retrieve features from Hopsworks.
- Train, evaluate, and register forecasting models.
- Generate 24-hour, 48-hour, and 72-hour AQI predictions.
- Display current and forecasted AQI through a dashboard.
- Provide AQI alerts and SHAP-based explanations.
- Automate feature processing and model retraining.

2.2 Non-Functional Requirements

- Reliability: Pipelines should run automatically with minimal manual intervention.
- Scalability: The system should support increasing amounts of historical data.
- Performance: Predictions and dashboard data should load within a reasonable time.
- Maintainability: Code should be modular and easy to update.
- Security: API keys and credentials should be stored securely.
- Usability: The dashboard should be simple and easy to understand.

2.3 Technical Requirements

- Programming: Python
- Data Processing: Pandas, NumPy
- Machine Learning: Scikit-learn, XGBoost
- Feature Store & Model Registry: Hopsworks
- Data Sources: OpenWeather, Open-Meteo
- Automation: GitHub Actions and Hopsworks Jobs

- Dashboard: Streamlit, Plotly
- Explainability: SHAP
- Deployment: Streamlit Community Cloud
- Version Control: Git and GitHub

3. System Overview

3.1 End-to-End System Architecture

The system follows an end-to-end ML architecture that collects weather and air-quality data, processes and stores features in Hopsworks, trains forecasting models, generates AQI predictions, and displays results through a Streamlit dashboard.

3.2 System Workflow

- Fetch weather and air-quality data.
- Perform feature engineering.
- Store features in Hopsworks Feature Store.
- Train models for 24h, 48h, and 72h forecasting.
- Register the trained models in Hopsworks Model Registry.
- Generate AQI forecasts.
- Display current and forecasted AQI on the Streamlit dashboard.
- Run automated feature collection and model training pipelines.

3.3 Project Components

- Feature Pipeline: Collects data and generates features.
- Historical Backfill: Generates historical training data.
- Training Pipeline: Trains and evaluates forecasting models.
- Prediction Pipeline: Generates 24h, 48h, and 72h AQI forecasts.
- Automated Pipelines: Hourly feature updates and daily model retraining.
- Streamlit Dashboard: Displays AQI, forecasts, EDA, alerts, and SHAP explanations.
- Hopsworks: Manages the Feature Store and Model Registry.

4. Data Collection

4.1 Weather Data Sources

Weather data was collected to provide environmental variables that can influence AQI. The main variables included temperature, relative humidity, surface pressure, and wind speed. Real-time weather data was obtained through the OpenWeather API. Historical weather data was collected using the Open-Meteo historical API. These features were later combined with air-quality data during feature engineering.

4.2 Air Quality Data Sources

Air-quality data was collected to obtain AQI and major pollutant concentrations. The collected pollutants included PM2.5, PM10, CO, NO_2, SO_2, and O_3. OpenWeather was used for current air-quality measurements. Open-Meteo was used to obtain historical hourly air-quality data. This data formed the main basis for AQI forecasting and analysis.

4.3 API Comparison and Findings

Multiple APIs were considered based on historical availability, real-time data, reliability, and ease of integration. OpenWeather provided convenient real-time weather and air-quality data but had limitations regarding historical data. Open-Meteo provided extensive historical hourly weather and air-quality data without requiring an API key. AQICN was also considered, but its historical data availability was less suitable for the project's training requirements. Based on these findings, OpenWeather and Open-Meteo were selected for different purposes.

4.4 OpenWeather API

OpenWeather was used primarily for obtaining current weather and air-quality information. The weather API provided temperature, humidity, pressure, and wind-speed measurements. Its air-pollution API provided pollutant concentrations and an AQI-related value. The API was integrated into the hourly feature pipeline to obtain updated environmental data. Its data was then processed and stored as features in Hopsworks.

4.5 Open-Meteo API

Open-Meteo was selected for collecting historical hourly weather and air-quality data. It provided sufficient historical coverage for creating the model-training dataset. Historical weather variables

included temperature, humidity, pressure, and wind speed. Historical air-quality variables included US AQI and major pollutant measurements. This data was used for historical backfilling, feature engineering, and model training.

4.6 AQI Data Considerations

A major consideration was ensuring that the AQI scale remained consistent across the entire system. OpenWeather's air-quality index uses a 1–5 scale, while the US AQI uses a 0–500 scale. These values cannot be directly treated as equivalent AQI measurements. During development, an inconsistency between these scales caused the current AQI to appear as 3 while forecasts were above 50. The issue was resolved by using consistent US AQI data throughout the feature, training, and prediction pipelines.

4.7 Final Data Source Selection

OpenWeather was selected for real-time weather and air-quality data used by the hourly pipeline. Open-Meteo was selected for historical weather and air-quality data used for training and backfilling. This combination provided both current observations and sufficient historical data. It also reduced dependency on a single API for all data requirements. The selected sources were integrated into the automated data pipeline.

4.8 Data Collection Frequency

The system is designed to collect and process new environmental data every hour. The AQI feature pipeline is automated through GitHub Actions and runs hourly. The hourly pipeline running as a Hopsworks Job also updates the Feature Store with current features. This ensures that recent AQI, weather, and pollutant information is continuously available. The updated data can then be used by the prediction and dashboard components.

4.9 Historical Data Collection

Historical data was collected at an hourly frequency using the Open-Meteo API to create the model-training dataset. The initial historical collection covered approximately 17,500 hourly records of weather and air-quality data. The data included weather variables, US AQI, and major pollutant measurements. The collected data was processed using the same feature-engineering logic used for current data. The resulting dataset was stored in Hopsworks and used for model training, evaluation, and persistence-baseline comparison.

5. Data Preprocessing

5.1 Data Cleaning

The collected weather and air-quality data was cleaned and converted into a consistent format before feature engineering. Unnecessary fields were removed, and relevant weather, pollutant, and AQI columns were retained. Numerical values were converted to appropriate numeric data types. The cleaned data was then prepared for feature generation and storage in Hopsworks.

5.2 Missing Values

Missing values were checked during the preprocessing and feature-engineering stages. Where appropriate, missing numerical values were handled using median-based filling for model explainability. Rows with unavailable target values were excluded during model training. This helped ensure that the models received valid and complete input data.

5.3 Timestamp Handling

All records contained a timestamp representing the observation time. Timestamps were converted into a consistent datetime format and sorted chronologically. Time-based features such as hour, day, month, and weekday were extracted from the timestamp. This allowed the models to capture temporal patterns in AQI.

5.4 Data Alignment

Weather and air-quality data were aligned using their corresponding timestamps. This ensured that measurements from different sources represented the same time period. The aligned datasets were then combined into a single feature dataset. Proper alignment was important for generating reliable relationships between environmental variables and AQI.

5.5 Duplicate Handling

The collected data was checked for duplicate records, particularly during historical backfilling and repeated pipeline runs. Records were organized according to their timestamps to maintain a consistent chronological dataset. Duplicate observations were removed where necessary to prevent repeated measurements from affecting model training. This helped maintain the quality and reliability of the Feature Store data.

5.6 Data Validation

The processed dataset was validated before being stored and used for training. Column names, data types, timestamps, AQI values, and pollutant measurements were checked for consistency. The US AQI values were also verified to ensure that the correct 0–500 AQI scale was being used. These validation steps helped identify data inconsistencies before they affected model predictions.

6. Feature Engineering

6.1 Time-Based Features

Extracted hour, day, month, and weekday from timestamps to capture temporal patterns in AQI.

6.2 Weather Features

Included temperature, humidity, surface pressure, and wind speed as environmental predictors.

6.3 Pollutant Features

Included PM2.5, PM10, CO, NO_2, SO_2, and O_3 concentrations as air-quality predictors.

6.4 AQI Difference

Calculated the difference between the current AQI and the previous AQI observation to capture short-term changes.

6.5 AQI Change Rate

Calculated the rate of AQI change between consecutive observations to identify the speed of AQI changes.

6.6 AQI Moving Average

Calculated a moving average of recent AQI values to capture the overall short-term AQI trend and reduce noise.

6.7 Rolling Temperature Average

Calculated a rolling average of temperature values to capture recent temperature patterns.

6.8 Temperature Difference

Calculated the change in temperature compared with the previous observation.

6.9 Humidity Difference

Calculated the change in humidity compared with the previous observation.

6.10 Final Feature Set

The final dataset combined weather, pollutant, time-based, and derived features. These features were used as inputs for the forecasting models.

7. Historical Data Backfill

7.1 Historical Data Collection Process

Historical hourly weather and air-quality data was collected using the Open-Meteo API to create the training dataset.

7.2 Backfill Pipeline

A historical backfill pipeline processed the collected data, generated features, and stored the resulting dataset in the Hopsworks Feature Store.

7.3 Dataset Size

The historical collection produced approximately 17,500 hourly records before further processing and target creation.

7.4 Training Dataset Preparation

The historical features were cleaned, validated, sorted chronologically, and prepared for model training. Rows without valid target values were removed where necessary.

7.5 Target Variable Creation

Future AQI values were created by shifting the AQI column forward according to each forecasting horizon. This allowed the models to learn how current conditions relate to future AQI.

7.6 Forecast Horizons

Three separate prediction targets were created for 24-hour, 48-hour, and 72-hour AQI forecasts. A separate model was trained and registered for each horizon.

8. Feature Store Implementation

8.1 Hopsworks Feature Store

Hopsworks Feature Store was used as the central cloud-based storage system for processed AQI features. It provided a structured and accessible location for both historical and continuously updated feature data.

8.2 Feature Group Design

A feature group named historical_aqi_features was created to store weather, pollutant, time-based, and derived AQI features. The same feature group supports both model training and prediction workflows.

8.3 Feature Group Schema

The schema contains weather features, pollutant measurements, AQI values, timestamps, time-based features, and derived features such as AQI change rate and moving averages. Consistent column names were maintained across the pipelines.

8.4 Feature Group Versioning

The feature group was maintained using version 1. Versioning allows the stored feature schema and data to be managed consistently across the project.

8.5 Data Ingestion

Processed features are inserted into Hopsworks after feature engineering. Historical data was backfilled initially, while new features are continuously added through the automated hourly pipeline.

8.6 Feature Retrieval

The prediction pipeline retrieves the latest feature records from Hopsworks and uses them as inputs to the registered forecasting models. The Streamlit dashboard also retrieves feature data from the Feature Store.

8.7 Challenges with Feature Store Storage

Several Hopsworks issues were encountered, including Delta/HDFS insertion errors and Arrow Flight/Query Service connection errors while reading the Feature Group. These issues required troubleshooting dependencies, storage configuration, and pipeline execution environments.

8.8 Final Feature Store Architecture

Hopsworks serves as the central Feature Store for the system. The GitHub Actions feature pipeline processes scheduled data, while the Hopsworks hourly job maintains current features. The training and prediction pipelines retrieve the processed features from Hopsworks for model development and forecasting.

9. Machine Learning Approach

9.1 Problem Formulation

The problem was formulated as a time-series regression task to predict future AQI values using historical weather, pollutant, temporal, and derived features.

9.2 Forecasting Horizons

Three forecasting horizons were used: 24 hours, 48 hours, and 72 hours ahead. Separate models were trained for each horizon.

9.3 Train/Test Split

The dataset was split chronologically into training and testing sets to preserve the time-series order and avoid data leakage from future observations.

9.4 Evaluation Metrics

9.4.1 MAE

Mean Absolute Error (MAE) measures the average absolute difference between predicted and actual AQI values. Lower values indicate better performance.

9.4.2 RMSE

Root Mean Squared Error (RMSE) measures prediction error while giving more weight to larger errors. Lower values indicate better performance.

9.4.3 R^2

R^2 (R-squared) measures how well the model explains variation in AQI values. Higher values indicate better performance.

9.5 Baseline Approach

A baseline was established to determine whether the machine learning models provided meaningful improvement over a simple prediction method.

9.6 Persistence Baseline Methodology

The persistence baseline predicts future AQI using the most recent available AQI value. Its MAE, RMSE, and R^2 were calculated for all three forecasting horizons and compared with the trained models.

10. Machine Learning Models Evaluated

10.1 Random Forest

Random Forest regression was evaluated as a tree-based ensemble model capable of capturing nonlinear relationships between environmental features and AQI.

10.2 Ridge Regression

Ridge Regression was evaluated as a regularized linear model. It performed well for the longer forecasting horizons and was selected for the 48-hour and 72-hour models.

10.3 Gradient Boosting / HistGradientBoosting

Gradient Boosting methods were tested to capture nonlinear relationships and interactions between features while maintaining good predictive performance.

10.4 XGBoost

XGBoost was evaluated as a gradient-boosted tree model. It achieved the best performance for the 24-hour forecasting horizon and was selected as the final 24-hour model.

10.5 Other Models Experimented With

Additional models were experimented with during development to compare different approaches and identify suitable algorithms for each forecasting horizon.

10.6 Model Comparison

Models were compared using MAE, RMSE, and R^2, alongside the persistence baseline. The final selected models were XGBoost for 24 hours and Ridge Regression for 48 and 72 hours.

Model	24h MAE	24h RMSE	24h R^2	48h MAE	48h RMSE	48h R^2	72h MAE	72h RMSE	72h R^2
Random Forest	6.62	9.82	0.4507	9.38	13.25	-0.0015	10.25	14.19	-0.1439
Hist. Gradient Boosting	6.27	9.27	0.5108	8.53	12.47	0.1125	9.68	13.70	-0.0667
Ridge	6.46	9.35	0.5027	8.66	12.05	0.1711	9.18	12.88	0.0577
XGBoost	6.17	9.21	0.5174	8.49	12.60	0.0935	10.11	14.21	-0.1460
Persistence Baseline	7.26	10.84	0.3316	10.28	14.59	-0.2153	11.83	16.54	-0.5544

11. Persistence Baseline Results

11.1 24-Hour Persistence Baseline

The persistence baseline for the 24-hour horizon achieved an MAE of 7.26, RMSE of 10.84, and R^2 of 0.3316.

11.2 48-Hour Persistence Baseline

For the 48-hour horizon, the persistence baseline achieved an MAE of 10.28, RMSE of 14.59, and R^2 of -0.2153.

11.3 72-Hour Persistence Baseline

For the 72-hour horizon, the persistence baseline achieved an MAE of 11.83, RMSE of 16.54, and R^2 of -0.5544.

11.4 Persistence MAE

MAE increased as the forecasting horizon increased, showing that simply carrying the current AQI forward becomes less accurate for longer-term predictions.

11.5 Persistence RMSE

RMSE also increased from 24 to 72 hours, indicating larger prediction errors at longer horizons.

11.6 Persistence R^2

The 24-hour baseline had a positive R^2, while the 48-hour and 72-hour baselines had negative R^2, showing weaker performance for longer horizons.

12. Final Model Results

12.1 24-Hour Model

XGBoost was selected for the 24-hour forecast, achieving an MAE of 6.17, RMSE of 9.21, and R^2 of 0.5174.

12.2 48-Hour Model

Ridge Regression was selected for the 48-hour forecast, achieving an MAE of 8.66, RMSE of 12.05, and R^2 of 0.1711.

12.3 72-Hour Model

Ridge Regression was selected for the 72-hour forecast, achieving an MAE of 9.18, RMSE of 12.88, and R^2 of 0.0577.

12.4 Model MAE

All three selected models achieved lower MAE than their corresponding persistence baselines, demonstrating improved prediction accuracy.

12.5 Model RMSE

The selected models also achieved lower RMSE values than the persistence baselines across all three horizons.

12.6 Model R^2

The selected models produced higher R^2 values than the persistence baseline for all horizons, with the largest improvement observed for the 24-hour forecast.

12.7 Comparison Against Persistence Baseline

The machine learning models outperformed the persistence baseline across MAE, RMSE, and R^2 for all three forecasting horizons.

Horizon	Persistence MAE	Selected Model	Model MAE	Persistence RMSE	Model RMSE	Persistence R^2	Model R^2
24h	7.26	XGBoost	6.17	10.84	9.21	0.3316	0.5174
48h	10.28	Ridge	8.66	14.59	12.05	-0.2153	0.1711
72h	11.83	Ridge	9.18	16.54	12.88	-0.5544	0.0577

12.8 Final Model Selection

The final models were XGBoost for 24 hours and Ridge Regression for both 48 and 72 hours.

12.9 Reasons for Selecting Different Models per Horizon

Different models were selected because model performance varied across forecasting horizons. XGBoost performed best for the short-term 24-hour prediction, while Ridge Regression provided better results for the longer 48-hour and 72-hour forecasts.

13. Model Training Pipeline

13.1 Training Workflow

The training pipeline retrieves historical features from Hopsworks, generates forecasting targets, trains multiple machine learning models, evaluates their performance, and registers the selected models.

13.2 Automated Training Process

The training process is automated through a scheduled Hopsworks job, reducing the need for manual model training and ensuring the system can regularly update its models.

13.3 Daily Model Retraining

The model training pipeline is configured to run daily. New historical data can therefore be incorporated into the training process and updated models can be registered.

13.4 Target Generation

Separate target variables are generated for 24-hour, 48-hour, and 72-hour AQI forecasts by shifting the AQI values according to each forecasting horizon.

13.5 Model Evaluation

Each trained model is evaluated using MAE, RMSE, and R^2. The results are compared to identify the most suitable model for each forecasting horizon.

13.6 Model Registration

The selected models are saved and registered in the Hopsworks Model Registry after training and evaluation.

13.7 Model Versioning

Each registered model is assigned a version number. This allows updated models to be stored while maintaining access to previous versions.

14. Model Registry

14.1 Hopsworks Model Registry

The Hopsworks Model Registry is used to centrally store, manage, and retrieve the trained AQI forecasting models.

14.2 Registered Models

Three forecasting models are registered:

- aqi_forecast_24h — XGBoost
- aqi_forecast_48h — Ridge Regression
- aqi_forecast_72h — Ridge Regression

14.3 Model Versions

The current deployed models are registered as Version 1 for each forecasting horizon. Future retraining can produce newer versions.

14.4 Model Artifacts

Each registered model contains the trained model artifact required to perform AQI predictions.

14.5 Model Retrieval for Prediction

The prediction pipeline retrieves the registered models from Hopsworks Model Registry and uses them with the latest feature data to generate 24-hour, 48-hour, and 72-hour AQI forecasts.

15. Prediction Pipeline

15.1 Prediction Workflow

The prediction pipeline retrieves the latest AQI features from Hopsworks, loads the registered forecasting models, and generates AQI predictions for the next 24, 48, and 72 hours.

15.2 Latest Feature Retrieval

The latest available feature record is retrieved from the Hopsworks Feature Store and used as the input for forecasting.

15.3 Model Loading

The prediction pipeline loads the corresponding registered models from the Hopsworks Model Registry.

15.4 24-Hour Prediction

The registered XGBoost model generates the AQI forecast for the next 24 hours.

15.5 48-Hour Prediction

The registered Ridge Regression model generates the AQI forecast for the next 48 hours.

15.6 72-Hour Prediction

The registered Ridge Regression model generates the AQI forecast for the next 72 hours.

15.7 Prediction Sanity Checks

Sanity checks were performed to verify that the retrieved features, current AQI, and forecast values were reasonable and consistent with the expected AQI scale.

15.8 Forecast Output

The generated predictions are displayed on the Streamlit dashboard along with the current AQI, AQI category, and forecast information.

16. Automated Pipelines

16.1 Overall Automation Architecture

The system uses three automated pipelines to handle feature processing, hourly updates, and model retraining with minimal manual intervention.

16.2 AQI Feature Pipeline — GitHub Actions

The AQI Feature Pipeline runs through GitHub Actions on an hourly schedule. It fetches the required data, performs feature engineering, and updates the Feature Store.

16.3 Hourly Pipeline — Hopsworks

The hourly pipeline runs as a Hopsworks Job and maintains the latest AQI feature data in the Feature Store.

16.4 Daily Model Training — Hopsworks

The model training pipeline runs daily on Hopsworks, retrieves the latest historical features, retrains the forecasting models, evaluates them, and registers the updated models.

16.5 Pipeline Scheduling

The feature pipeline is scheduled to run hourly, while the model training pipeline is scheduled to run daily.

16.6 Pipeline Dependencies

The training pipeline depends on the availability of processed historical features, while the prediction pipeline depends on the latest features and registered models.

16.7 Automation Workflow

The overall workflow is: data collection → feature engineering → Feature Store → model training → Model Registry → prediction → dashboard.

16.8 Failure Handling / Monitoring

Pipeline execution logs are used to identify failures and monitor successful runs. Issues encountered during development were investigated and resolved through dependency, storage, and execution-environment changes.

17. CI/CD Implementation

17.1 GitHub Actions

GitHub Actions was used to automate the AQI feature pipeline and execute scheduled data processing without requiring manual execution.

17.2 Workflow Configuration

A YAML workflow file defines the pipeline schedule, Python environment, dependency installation, required secrets, and execution command.

17.3 Secrets Management

Sensitive API credentials such as OpenWeather and Hopsworks API keys are stored as GitHub Actions secrets rather than being directly written into the source code.

17.4 Automated Feature Pipeline

The GitHub Actions workflow automatically executes the AQI feature pipeline every hour, ensuring that new processed feature data is continuously generated.

17.5 Deployment Workflow

The application code is maintained in GitHub and deployed through Streamlit Community Cloud. Updates pushed to the repository can be used to update the deployed dashboard.

18. Dashboard Development

18.1 Streamlit Dashboard

A web-based interactive dashboard was developed using Streamlit to present current AQI, forecasts, historical data, analytics, and model performance in one interface.

18.2 Dashboard Architecture

The dashboard retrieves the latest features and registered models, processes predictions, and presents the results through interactive visualizations and metrics.

18.3 Real-Time AQI Display

The dashboard displays the latest available AQI along with its corresponding air-quality category.

18.4 24/48/72-Hour Forecasts

Forecasted AQI values for 24, 48, and 72 hours are displayed using the trained and registered forecasting models.

18.5 Historical AQI Visualization

Historical AQI data is visualized to show changes and patterns in air quality over time.

18.6 Pollutant Levels

The dashboard displays major pollutant measurements including PM2.5, PM10, CO, NO_2, and SO_2.

18.7 EDA & Trends

Exploratory visualizations were included to analyze AQI behavior, pollutant patterns, and environmental trends.

18.8 AQI Distribution

The distribution of historical AQI values is visualized to understand the frequency and range of different AQI levels.

18.9 Hourly and Monthly Analysis

Hourly and monthly visualizations were implemented to identify recurring temporal patterns in AQI.

18.10 Pollutant Trends

Historical trends of individual pollutants are displayed to observe their changes over time and their relationship with AQI.

18.11 Correlation Analysis

Correlation analysis was used to examine relationships between AQI and the available weather and pollutant features.

18.12 Prediction Explainability

Model explainability was included to provide insight into which features contribute to the forecasting results.

18.13 SHAP Analysis

SHAP (SHapley Additive exPlanations) was used to analyze feature importance and understand the contribution of individual features to model predictions.

18.14 Dynamic Model Performance Table

A dynamic table displays the performance of the selected models for each forecasting horizon using MAE, RMSE, and R^2, along with their persistence baseline results.

18.15 AQI Alerts

The dashboard generates alerts when the current or forecasted AQI reaches a hazardous category, helping users identify potentially unhealthy air-quality conditions.

18.16 EPA AQI Scale

The dashboard uses the EPA AQI scale to categorize AQI values into levels such as Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, and Hazardous.

19. Dashboard Deployment

19.1 Streamlit Community Cloud

The completed Streamlit dashboard was deployed using Streamlit Community Cloud, making the application accessible through a public web URL.

19.2 Deployment Configuration

The repository, main Streamlit application file, Python dependencies, and required configuration were configured for deployment.

19.3 Secrets / secrets.toml

API keys and other sensitive configuration values were added through Streamlit's secrets management using the secrets.toml format instead of exposing credentials in the source code.

19.4 Environment Variables

The application uses environment-based configuration to access API credentials and Hopsworks settings without hardcoding sensitive information.

19.5 Production Testing

After deployment, the dashboard was tested in the production environment to verify that data retrieval, predictions, visualizations, model performance values, and AQI alerts were functioning correctly.

19.6 Final Deployment URL

The final Streamlit Community Cloud deployment URL provides access to the completed AQI forecasting dashboard.

20. Findings and Observations

20.1 API Comparison Findings

Open-Meteo was found to be more suitable for historical weather and air-quality data because it provides accessible hourly historical data. OpenWeather was used for current weather and air-quality information.

20.2 Data Quality Findings

The collected data required validation to ensure consistent timestamps, feature names, AQI values, and pollutant measurements. An important issue was identified with inconsistent AQI scales, which was resolved by recreating the Hopsworks Feature Store.

20.3 Feature Engineering Findings

Time-based, weather, pollutant, and derived AQI features helped provide the models with information about both current conditions and recent trends.

20.4 Model Performance Findings

The selected machine learning models outperformed the persistence baseline for all three forecasting horizons. XGBoost performed best for 24-hour forecasting, while Ridge Regression performed better for the longer horizons.

20.5 Persistence Baseline Findings

The persistence baseline became less accurate as the forecasting horizon increased. Its MAE increased from 7.26 for 24h to 11.83 for 72h, demonstrating the difficulty of simply carrying the current AQI forward.

20.6 Forecast Horizon Findings

Prediction accuracy decreased as the forecast horizon increased. The 24-hour forecast achieved the strongest results, while the 72-hour forecast was more challenging.

20.7 Important Predictive Features

Feature importance and SHAP analysis were used to identify the features contributing most to predictions. Recent AQI values, pollutant measurements, and environmental conditions were among the important predictive factors.

20.8 Differences Between 24h, 48h and 72h Forecasting

The 24-hour horizon showed the strongest predictive performance, while 48-hour and 72-hour predictions were more difficult due to greater uncertainty in future environmental conditions.

20.9 Why Model Performance Changes With Horizon

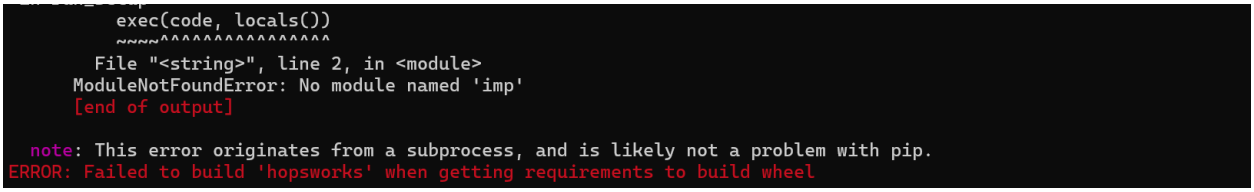
As the forecasting horizon increases, the relationship between current conditions and future AQI becomes weaker. Changes in weather and pollutant concentrations introduce additional uncertainty, making longer-term predictions more difficult.

20.10 Operational Findings

Automation significantly reduced manual intervention. The final system uses three automated pipelines: the AQI Feature Pipeline through GitHub Actions, the hourly pipeline through Hopsworks, and daily model training through Hopsworks.

21. Challenges, Blockers and Resolutions

Error 1: Hopsworks Installation Failure on Windows (twofish Dependency)



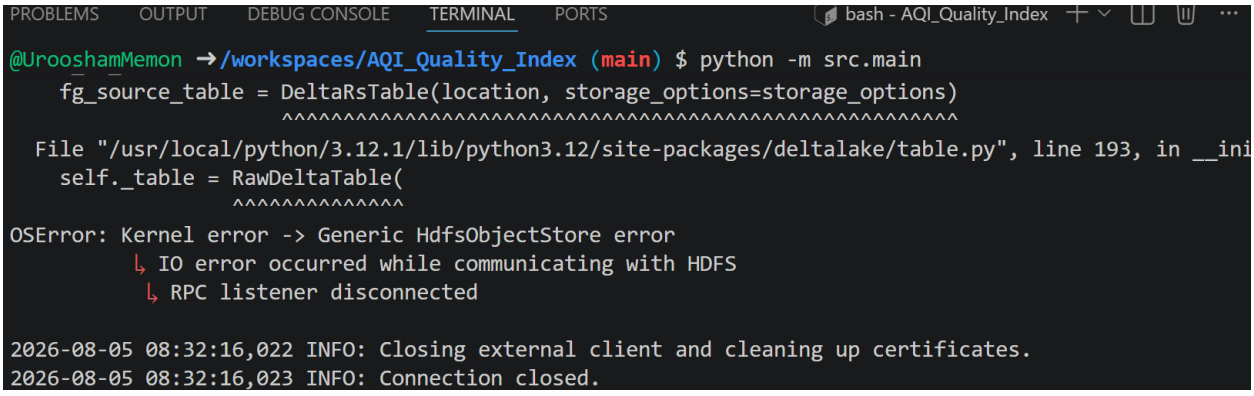
Problem

While installing Hopsworks on the local Windows environment, an installation error occurred with the twofish dependency. The package attempted to install using the legacy setup.py install method and failed during the installation process. This prevented Hopsworks from being installed and used reliably in the local Windows environment.

Solution

To resolve the compatibility issue, GitHub Actions was used to run the Hopsworks-related pipeline in an Ubuntu environment instead of Windows. Since GitHub Actions provides a Linux-based runner, it avoided the Windows-specific twofish installation problem and allowed Hopsworks to be installed and used successfully. This approach also provided a more suitable environment for running the automated feature pipeline.

Error 2: Feature Store Data Insertion Error (Delta/HDFS)



Problem

After successfully creating the Feature Group in Hopsworks, an error occurred while inserting feature data into the Feature Store. Initially, it was suspected that the error was caused by the large volume of data being uploaded. To verify this, the upload was tested with a smaller dataset of 100

rows, but the same error occurred. The test was then reduced to a single row, and the exact same HDFS/Delta error was still produced.

This confirmed that the issue was not related to the dataset size or the DataFrame itself, but was instead related to the Hopsworks storage layer and its Delta/HDFS dependencies.

Solution

The issue was resolved by installing the optional Python dependencies required by Hopsworks for proper Delta/HDFS operations. The standard pip install hopsworks installation did not include all the dependencies required for the Feature Store upload to function correctly.

The existing Hopsworks installation was first removed: pip uninstall hopsworks -y

Hopsworks was then reinstalled with its additional Python dependencies: pip install --upgrade "hopsworks[python]"

After installing the required dependencies, the Feature Store upload process was able to proceed correctly.

Error 3: Hopsworks Errors Blocking Local Pipeline Testing

```
if enable_hopsworks:
    store_on_hopsworks(history_df)
else:
    print("Hopsworks disabled. Running on local server")
```

Problem

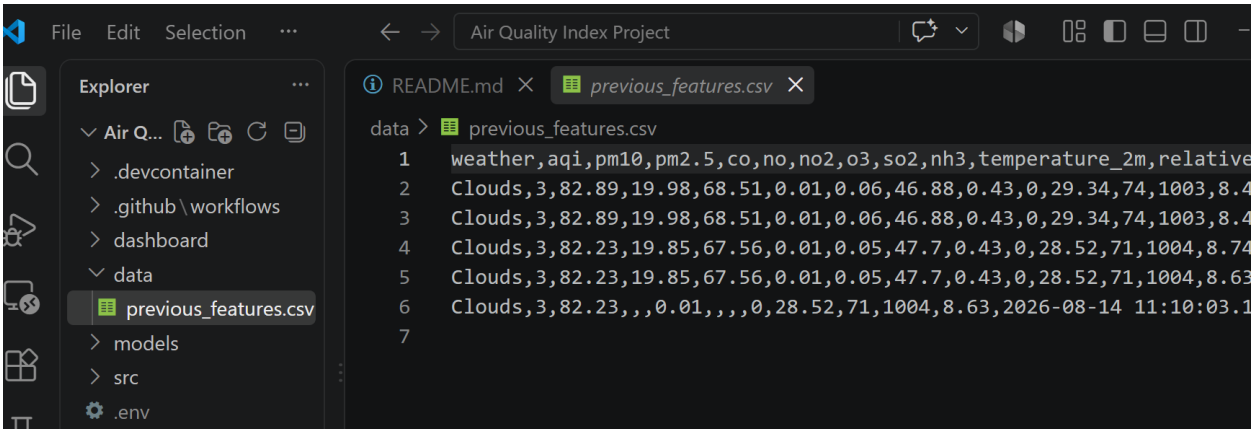
While developing and testing the pipeline locally, Hopsworks-related errors were preventing the entire pipeline from running successfully. This made it difficult to test the data fetching and feature engineering components independently of Hopsworks.

Solution

An environment flag, ENABLE_HOPSWORKS, was introduced to control Hopsworks functionality based on the execution environment. It was set to False in the local environment and True in GitHub Codespaces, allowing Hopsworks to be disabled during local development while remaining enabled in the deployment environment.

Additionally, the pipeline was modified so that when Hopsworks is disabled, the data fetching and feature engineering stages continue to execute instead of causing the entire pipeline to fail. This allowed individual components to be tested locally without requiring a Hopsworks connection, while the complete pipeline could still run with Hopsworks enabled in GitHub Codespaces.

Error 4: Limited Local Testing for Components Dependent on previous_df



Problem

When Hopsworks was disabled in the local environment, the pipeline could continue with data fetching and feature engineering, but components that depended on previous_df could not be tested in the same way as they were in the Hopsworks environment. This created a difference between local testing and the actual automated pipeline.

Solution

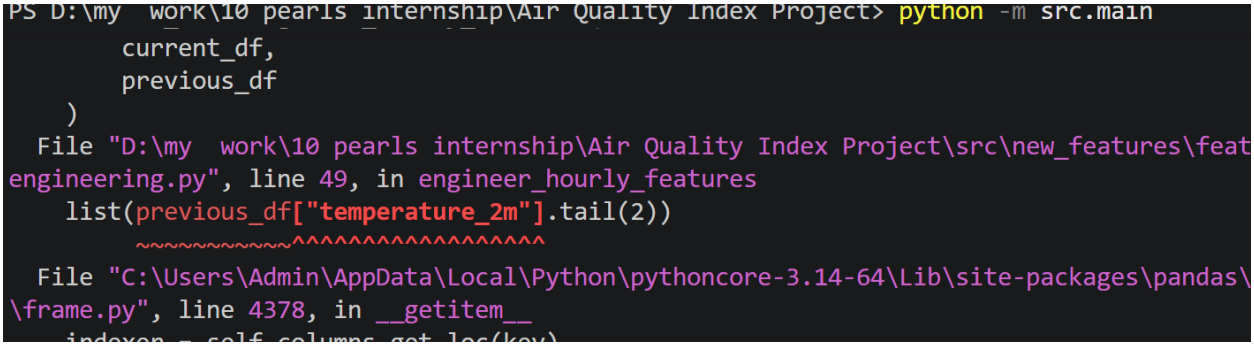
A local fallback approach was introduced/planned for previous_df so that a small number of recent feature records could be stored locally in a CSV. In the local environment, this file can be used as the previous historical data when Hopsworks is disabled, while Codespaces and GitHub Actions continue to retrieve the previous data from Hopsworks.

This allows both environments to use the same feature-engineering logic, with only the data source changing:

- Local: Recent records from CSV
- Codespaces/GitHub Actions: Historical records from Hopsworks

This approach improves local testing and reduces dependency on a live Hopsworks connection during development.

Error 5: Column Name Mismatch Between Hourly and Historical Features



Problem

During the development of the hourly feature pipeline, it was identified that the hourly features were using different column names from those defined in the historical Feature Group. This inconsistency could cause problems when combining hourly data with historical data and when passing the features to the trained models, which expect a specific feature schema.

Solution

The create_features() function was updated to rename the hourly feature columns so that they exactly match the column names used in the historical Feature Group. This ensured a consistent feature schema across the historical data, hourly pipeline, model training, and prediction stages, reducing the risk of column-mismatch errors during automated execution.

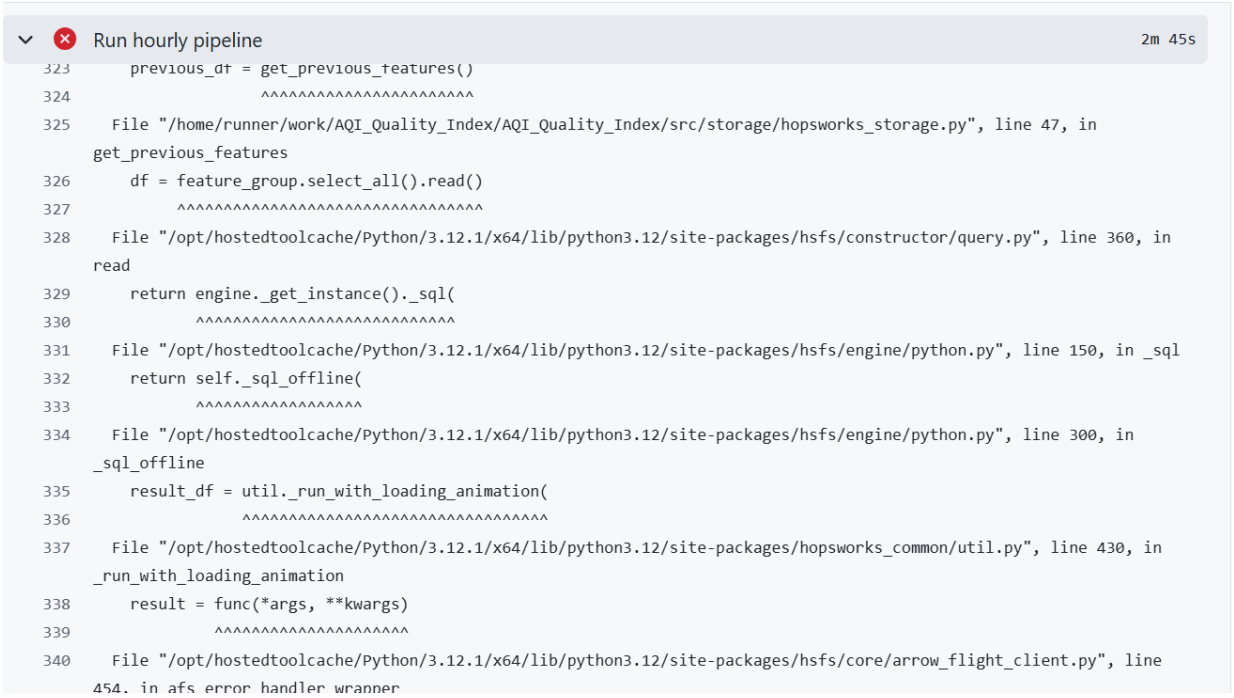
```python
def create_features():
    weather_features = fetch_weather_data()
    air_features = fetch_air_data()

    features = weather_features | air_features

    features["temperature_2m"] = features.pop("temperature")
    features["relative_humidity_2m"] = features.pop("humidity")
    features["surface_pressure"] = features.pop("pressure")
    features["wind_speed_10m"] = features.pop("wind_speed")
    features["pm2_5"] = features.pop("pm2.5")
    features["carbon_monoxide"] = features.pop("co")
    features["nitrogen_dioxide"] = features.pop("no2")
    features["sulphur_dioxide"] = features.pop("so2")
    features["ozone"] = features.pop("o3")

    return features
```

Error 6: Delta/HDFS Write Error in the GitHub Actions Hourly Pipeline



```
Run hourly pipeline                                                    2m 45s
323     previous_df = get_previous_features()
324                   ^^^^^^^^^^^^^^^^^^^^^^^^
325   File "/home/runner/work/AQI_Quality_Index/AQI_Quality_Index/src/storage/hopsworks_storage.py", line 47, in
    get_previous_features
326     df = feature_group.select_all().read()
327          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
328   File "/opt/hostedtoolcache/Python/3.12.1/x64/lib/python3.12/site-packages/hsfs/constructor/query.py", line 360, in
    read
329     return engine._get_instance()._sql(
330            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
331   File "/opt/hostedtoolcache/Python/3.12.1/x64/lib/python3.12/site-packages/hsfs/engine/python.py", line 150, in _sql
332     return self._sql_offline(
333            ^^^^^^^^^^^^^^^^^^
334   File "/opt/hostedtoolcache/Python/3.12.1/x64/lib/python3.12/site-packages/hsfs/engine/python.py", line 300, in
    _sql_offline
335     result_df = util._run_with_loading_animation(
336                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
337   File "/opt/hostedtoolcache/Python/3.12.1/x64/lib/python3.12/site-packages/hopsworks_common/util.py", line 430, in
    _run_with_loading_animation
338     result = func(*args, **kwargs)
339              ^^^^^^^^^^^^^^^^^^^^^
340   File "/opt/hostedtoolcache/Python/3.12.1/x64/lib/python3.12/site-packages/hsfs/core/arrow_flight_client.py", line
    454, in afs_error_handler_wrapper
```

Problem

While running the hourly pipeline through GitHub Actions, an error occurred during the data insertion stage. The traceback showed that the failure was occurring specifically in the

Delta/HDFS write path. The Hopsworks operation was selecting the local Delta-RS path, which resulted in an HDFS-related connection error and prevented the hourly feature data from being successfully written to the Feature Store.

It also had another issue, that occurred before any new data was inserted. The function was attempting to retrieve the existing Feature Group data using: feature_group.select_all().read()

Solution

Since the issue persisted in the GitHub Actions environment, the hourly feature pipeline was shifted from GitHub Actions to a Hopsworks Job. The Hopsworks Job runs the pipeline directly within the Hopsworks environment, avoiding the problematic Delta/HDFS write path encountered through GitHub Actions.

This allowed the hourly pipeline to run reliably and automatically within Hopsworks while GitHub Actions was retained for the separate AQI Feature Pipeline.

Error 7: Inconsistent Current AQI Value in Prediction Output

Problem

During the initial testing of the prediction pipeline, the current AQI was reported as 3, while the 24-hour, 48-hour, and 72-hour forecasts were approximately 66.72, 66.60, and 72.36 respectively. The issue was traced to an inconsistency in the AQI source used by the feature pipeline. OpenWeather's Air Pollution API provides an AQI value on a 1–5 scale, whereas the us_aqi value from Open-Meteo uses the US AQI scale (0–500). The hourly feature pipeline was intended to use us_aqi, but the current AQI value being retrieved and stored was coming from the OpenWeather 1–5 scale. Therefore, the latest feature record contained an AQI value of 3, which was incorrectly interpreted as the current AQI on the US AQI scale. This resulted in a large difference between the displayed current AQI and the forecasted values.

Solution

To resolve the issue, a new Hopsworks Feature Store was created and the feature pipeline was rerun to populate it with fresh and consistent data using the correct us_aqi value. The forecasting models were then retrained using the data from the new Feature Store, and the best-performing

models were registered in the Model Registry. The prediction pipeline was subsequently tested using the new Feature Store and newly registered models.

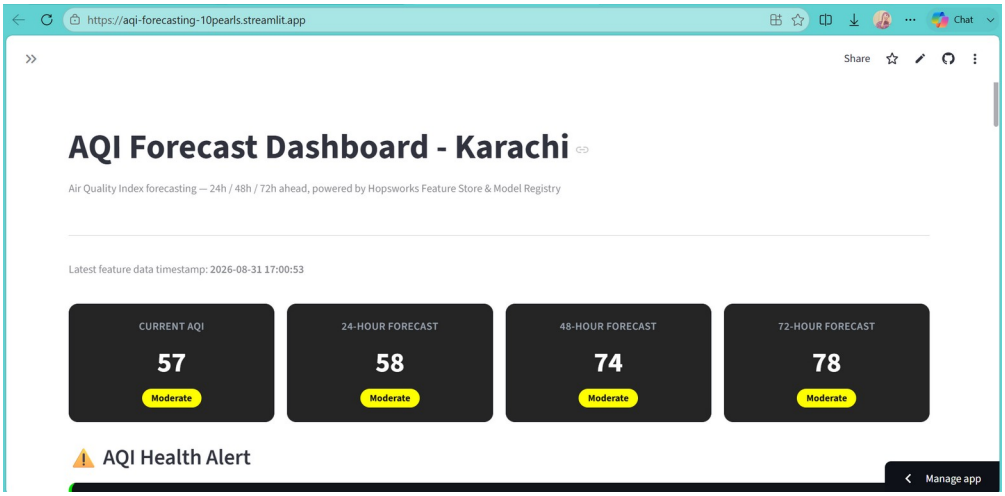
After this correction, the current AQI was reported as 61, with forecasts of 59.28 (24h), 71.20 (48h), and 74.26 (72h). This confirmed that the issue was caused by the mismatch between the OpenWeather 1–5 AQI scale and the US AQI scale, rather than an error in the forecasting model or prediction logic.

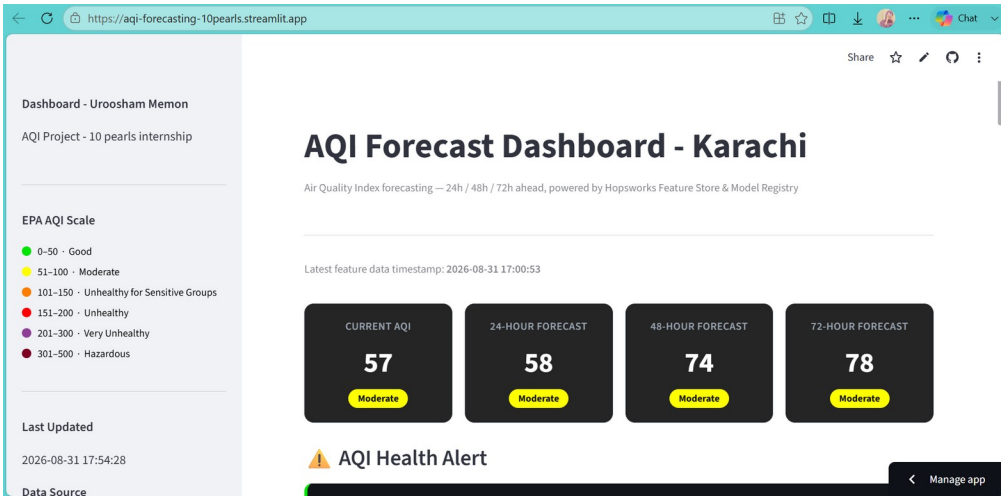
Lessons Learned

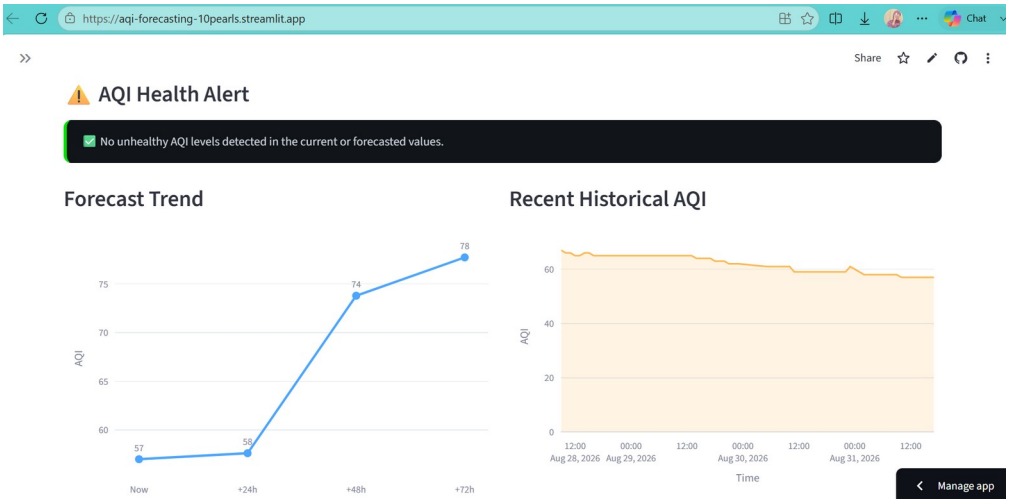
The development process highlighted the importance of environment compatibility, dependency management, cloud storage limitations, consistent data schemas, and proper secret management. It also demonstrated the value of testing individual pipeline components before integrating the complete system.

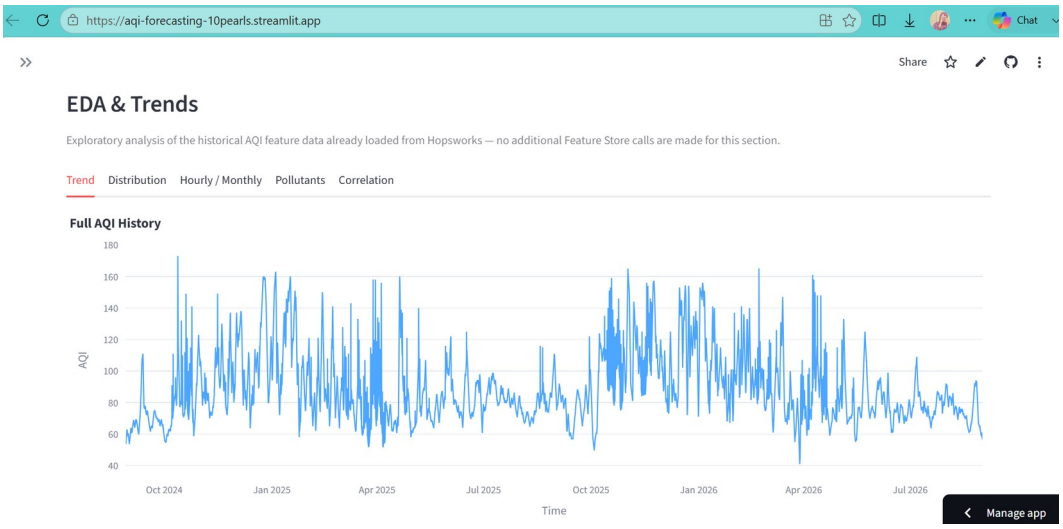
22. Dashboard Screenshots

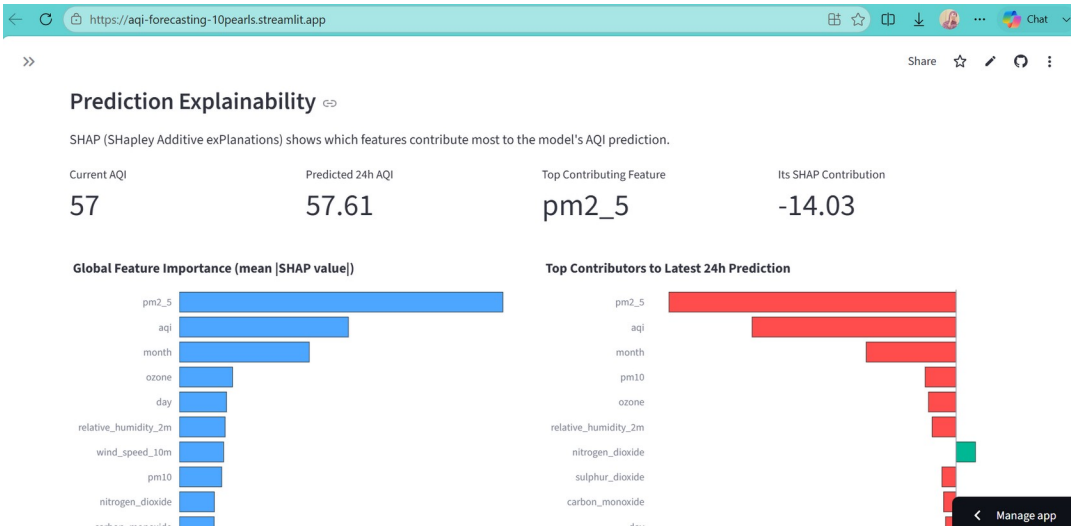
The following screenshots present the deployed Streamlit dashboard.

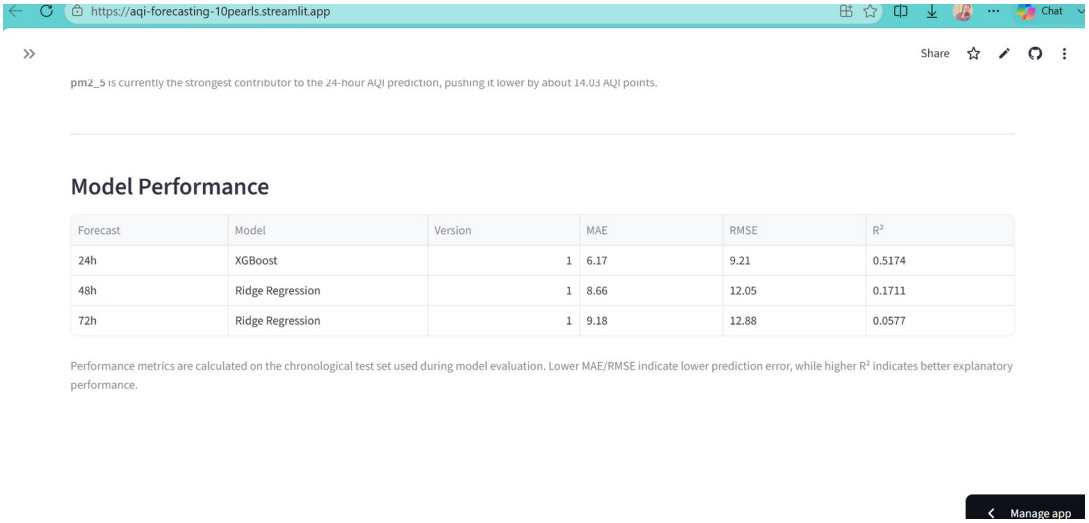







https://aqi-forecasting-10pearls.streamlit.app
Chat
Share
EDA & Trends
Exploratory analysis of the historical AQI feature data already loaded from Hopsworks — no additional Feature Store calls are made for this section.
Trend
Distribution
Hourly / Monthly
Pollutants
Correlation
Full AQI History
AQI
Time
Oct 2024
Jan 2025
Apr 2025
Jul 2025
Oct 2025
Jan 2026
Apr 2026
Jul 2026
Manage app


https://aqi-forecasting-10pearls.streamlit.app
Chat
Share
Prediction Explainability
SHAP (SHapley Additive exPlanations) shows which features contribute most to the model's AQI prediction.
Current AQI
57
Predicted 24h AQI
57.61
Top Contributing Feature
pm2_5
Its SHAP Contribution
-14.03
Global Feature Importance (mean |SHAP value|)
Top Contributors to Latest 24h Prediction
pm2_5
aqi
month
ozone
day
relative_humidity_2m
wind_speed_10m
pm10
nitrogen_dioxide
sulphur_dioxide
carbon_monoxide
Manage app


https://aqi-forecasting-10pearls.streamlit.app
Chat
Share
pm2_5 is currently the strongest contributor to the 24-hour AQI prediction, pushing it lower by about 14.03 AQI points.
Model Performance
Forecast
Model
Version
MAE
RMSE
R²
24h
XGBoost
1
6.17
9.21
0.5174
48h
Ridge Regression
1
8.66
12.05
0.1711
72h
Ridge Regression
1
9.18
12.88
0.0577
Performance metrics are calculated on the chronological test set used during model evaluation. Lower MAE/RMSE indicate lower prediction error, while higher R² indicates better explanatory performance.
Manage app

23. EDA and SHAP Visualizations

Figure 1:

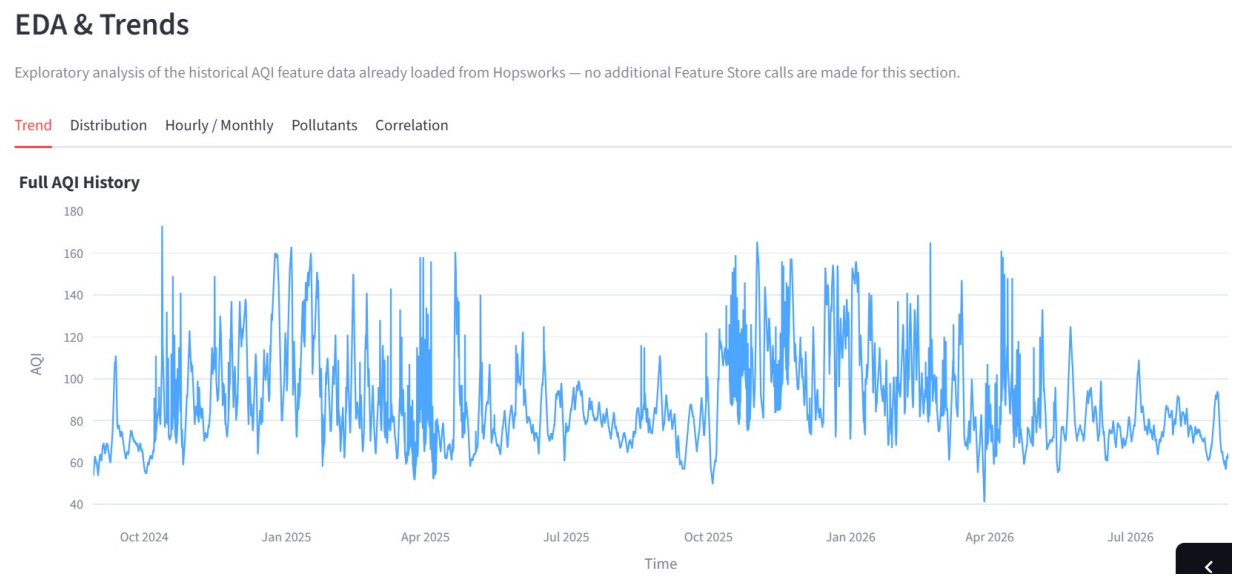


Figure 2:

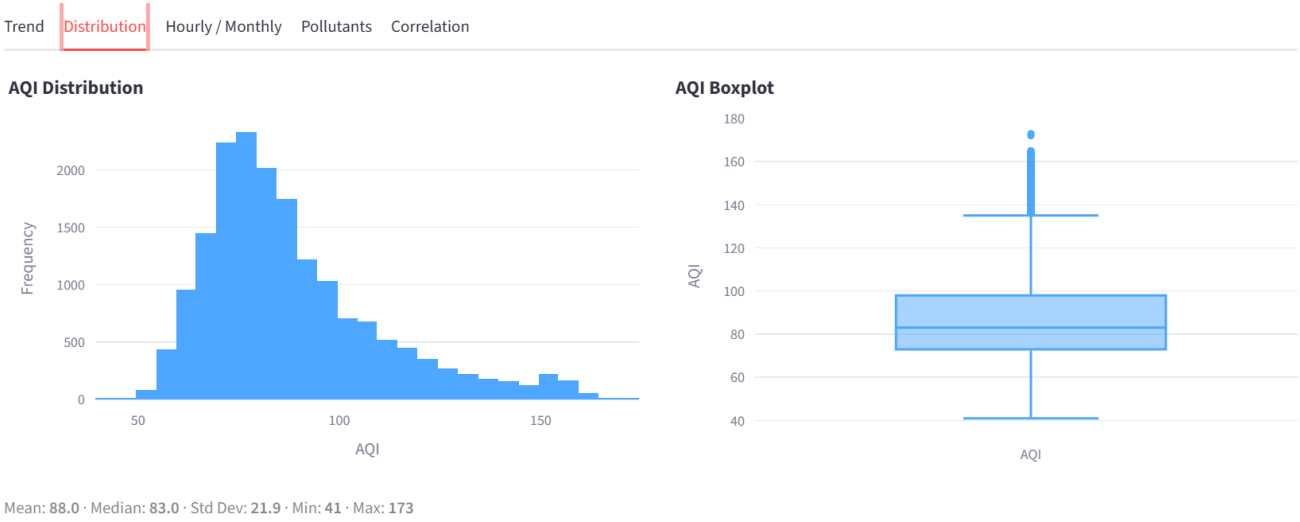


Figure 3:

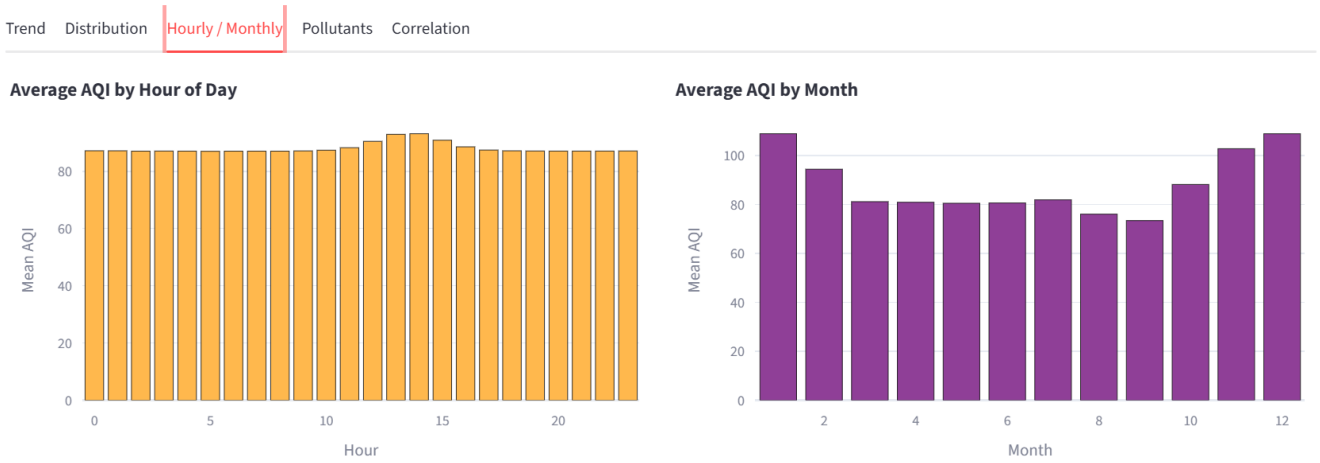


Figure 4:

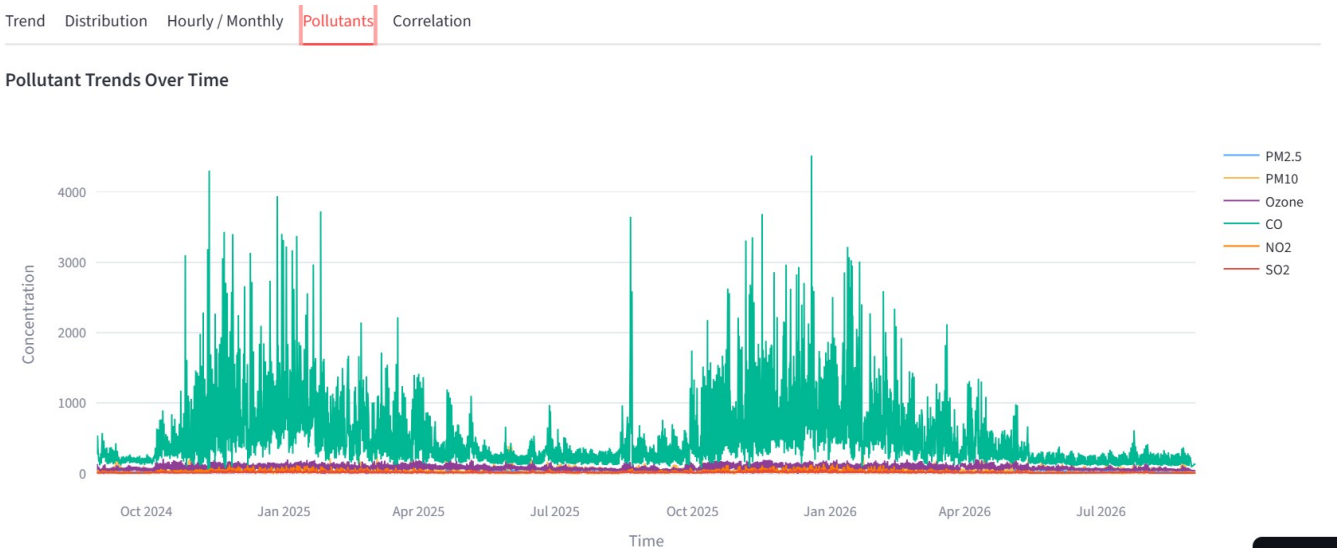


Figure 5:

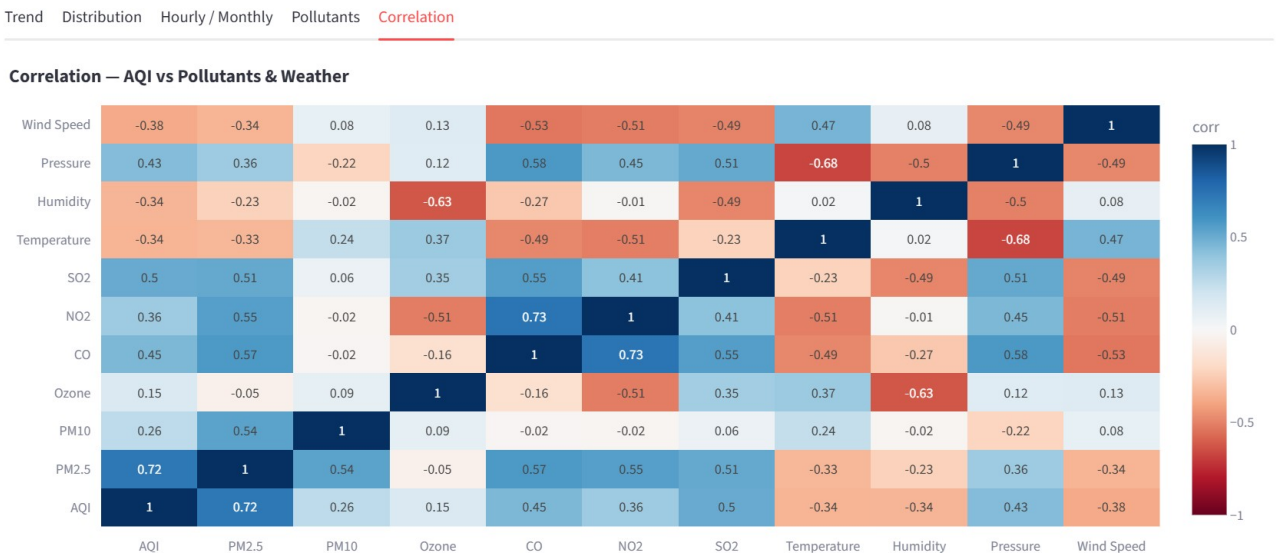


Figure 6:

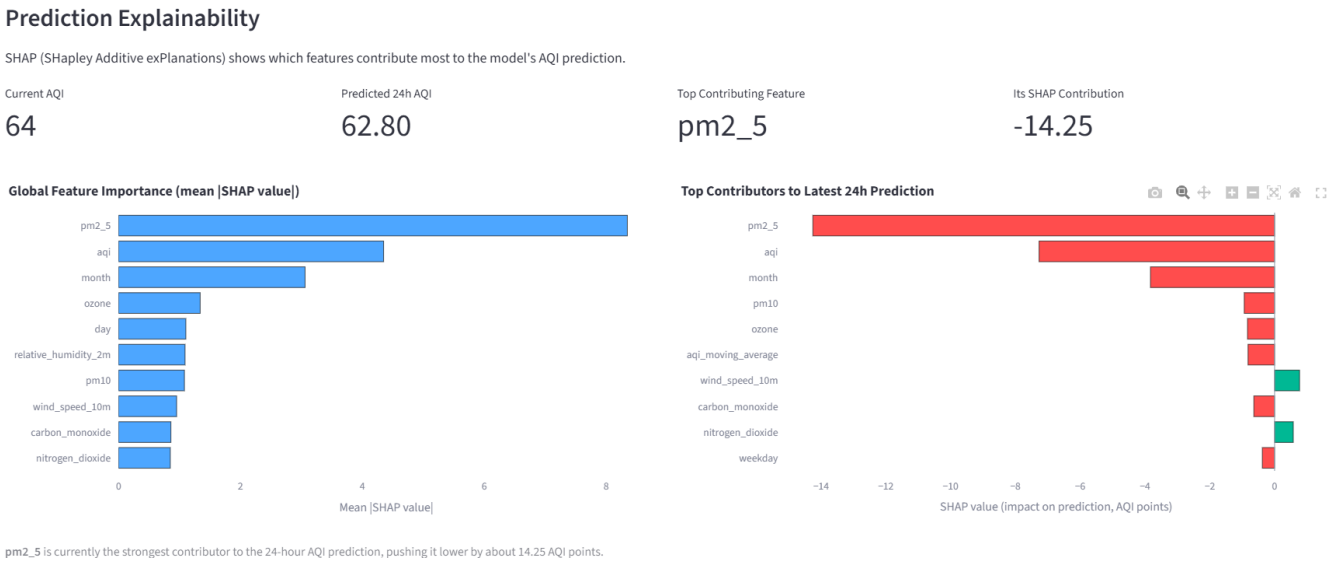


Figure 7:

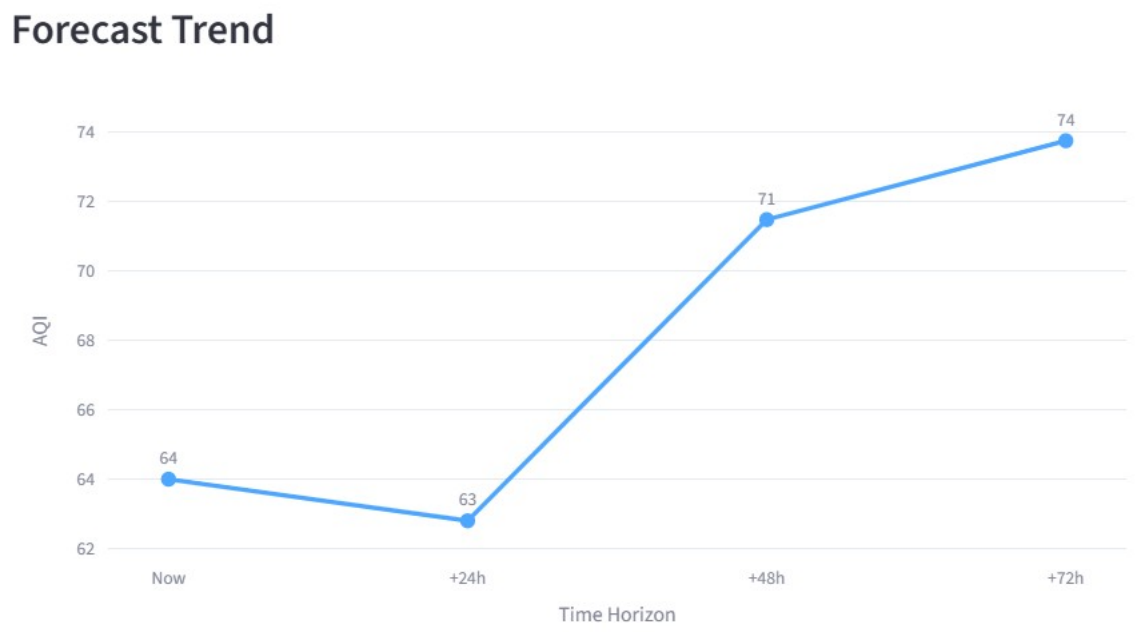


Figure 8: